TOP / 전체 학습 지도

FastAPI 예습 노트 · memvid 개정판

받고 돌려주기 → 업무 규칙 → 저장과 외부 호출 순서로 읽기

쉽게 말하면

Spring에서 승인한 사용자·로펌·사건 범위로 FastAPI가 요청을 처리합니다. 검색 계층은 사건별 memvid 메모리이며, 검색된 근거를 모델에 전달해 검토용 보고서를 만듭니다.

학습 구역	이 문서에서 읽을 것
TOP	라우터·본문 모델·Depends·내부 인증·상태 코드
MIDDLE	스크리닝·Dynamic Guard·예산·예약·이벤트·퍼널
BOTTOM	memvid 파일·임베딩·업무 DB·모델·테스트 대역
개정 범위	원본 0-9절 보존, 세로 도식 재작성, 저장·검색·테스트를 memvid 기준으로 변경

원문의 코드 경로·수치·테스트 사례는 과거 설명입니다. 새 메모리 모듈·예제·평가 항목은 개정 설계로 구분했으며 애플리케이션 구현 완료를 뜻하지 않습니다.

TOP / NAVIGATION

목차 · 어디부터 읽을까

목차 항목 또는 PDF 책갈피를 누르면 해당 주제로 이동합니다.

0 전체 구조

1 요청 한 번의 여행

2-A Depends 코드 읽기

2-B 인증·범위·수명

3-A 스크리닝 전처리

3-B 사후 검사

memvid 부품과 역할

memvid 코드 예제

4 Dynamic Guard

5-A 크레딧 상태

5-B 과금 예외

6-A 예약 상태 전이

6-B 퍼널 지표

7 도메인 이벤트

8-A 테스트 구조

8-B 테스트 목록

9-A 개념↔코드

9-B 응답·구조

연습 과제

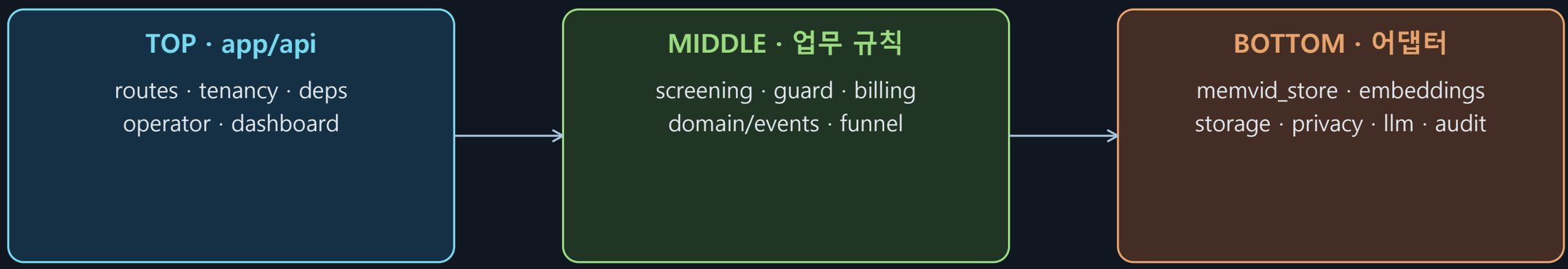
실행·검토 순서

TOP / LAYERS

0 · 전체 구조: TOP - MIDDLE - BOTTOM

쉽게 말하면

TOP은 HTTP를 받고 결과를 돌려주며, MIDDLE은 규칙과 순서를 결정합니다. BOTTOM은 DB·memvid·모델 같은 실제 부품을 실행합니다. 아래 계층은 FastAPI에 의존하지 않도록 설계합니다.



계층 예외를 설명하는 법	돈이 드는 모델 호출은 예산 정책이 적용되는 실행 경로만 사용합니다. 모듈 간 호출 방향보다 책임·인터페이스·우회 방지를 먼저 고정합니다.

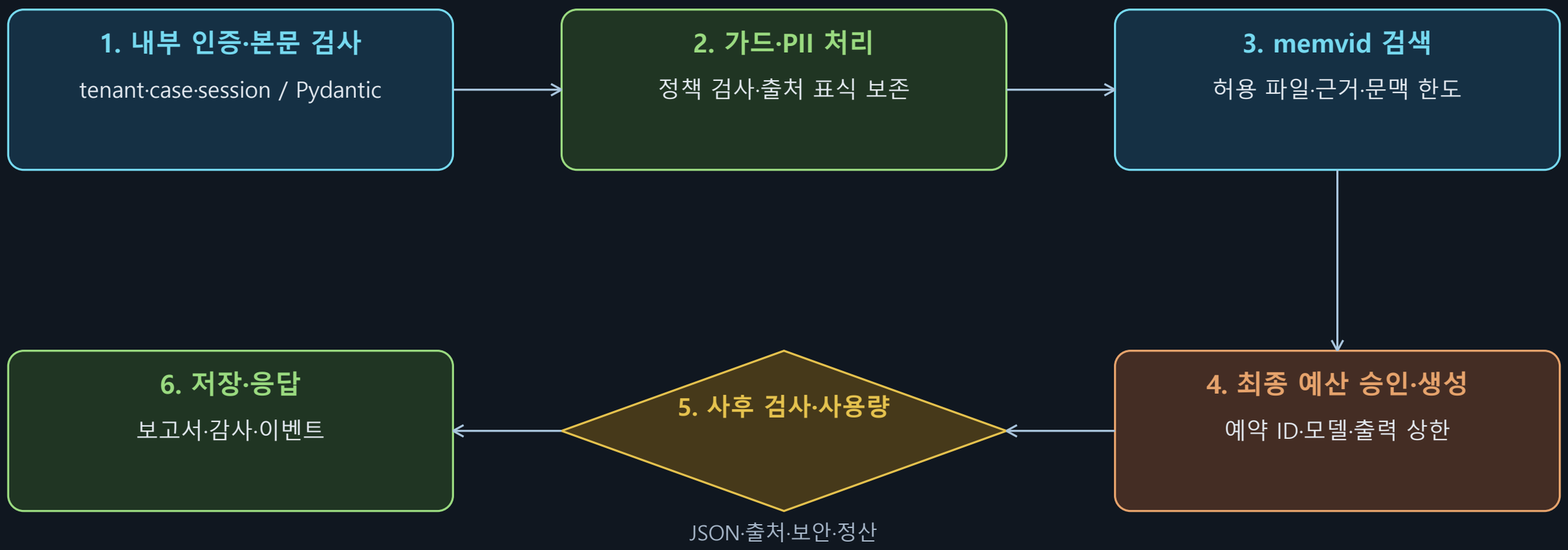
memvid_store·embeddings 모듈은 제안 이름입니다. 업무 DB는 보고서·예약·비용 기록을, memvid는 문서·검색 메모리를 맡습니다. 원문 감사 로그는 원문 대신 해시를 남기는 방향을 설명합니다.

MIDDLE / POST /v1/screenings

1 · 요청 한 번의 여행

쉽게 말하면

인증·본문 검증·가드처럼 상대적으로 저렴한 검사를 먼저 수행합니다. 유료 임베딩과 생성은 각각 승인된 예산 안에서 실행하며 사용량을 따로 남깁니다.



원문 응답 분류: 401 인증, 422 입력, 403 정책, 402 예산. 실제 실패 우선순위는 의존성 구성과 구현에 따르므로 HTTP 시험으로 확인합니다. 차단 전 이미 든 비용은 별개입니다.

TOP / DEPENDENCY INJECTION

2-A · Depends: 먼저 구한 값을 주입

쉽게 말하면

Depends는 이 함수가 필요로 하는 값을 다른 함수에서 받아 연결하는 장치입니다. 인증 문맥과 서비스 조립을 엔드포인트 본문에 섞지 않도록 돕습니다.

```
@router.post("/screenings")
def create_screening(
    situation: Situation,
    ctx: TenantContext = Depends(tenant_context),
    services: Services = Depends(get_services),
):
    return services.screening.run(ctx=ctx, situation=situation)
```

Situation: JSON 본문을 검증하는 모델. ctx: 검증된 인증·사건 문맥. services: 실행에 필요한 부품 묶음. 위 코드는 역할을 설명하는 시그니처이며 관련 타입·라우터 구현이 별도로 필요합니다.

공식 FastAPI 개념은 Depends이며, TenantContext·Services·get_services·deps.reset은 애플리케이션이 정의하는 이름입니다.

TOP / SECURITY & LIFETIME

2-B · 인증 문맥과 사건 메모리의 수명

쉽게 말하면

요청 본문이 주장하는 tenant_id나 파일 경로는 권한 근거가 아닙니다. Spring이 승인한 내부 문맥을 검증하고 서버 매핑으로 파일을 선택합니다.

관심사	memvid 개정 설계
내부 토큰	서명·만료·대상 서비스·역할·사건 범위를 검증. HMAC은 암호화와 구분
요청 본문	모델에 선언된 필드·길이·형식 검증. 추가 필드 무시/거부는 명시 설정
서비스 조립	공유 가능한 설정·공급자와 요청별 인증·파일 핸들을 구분
memvid 파일	사용자 입력 경로를 그대로 열지 않고 사건 매핑·허용 루트 확인
종료와 테스트	yield 의존성 또는 명시적 문맥으로 핸들 정리. 테스트 종료 시 override도 정리

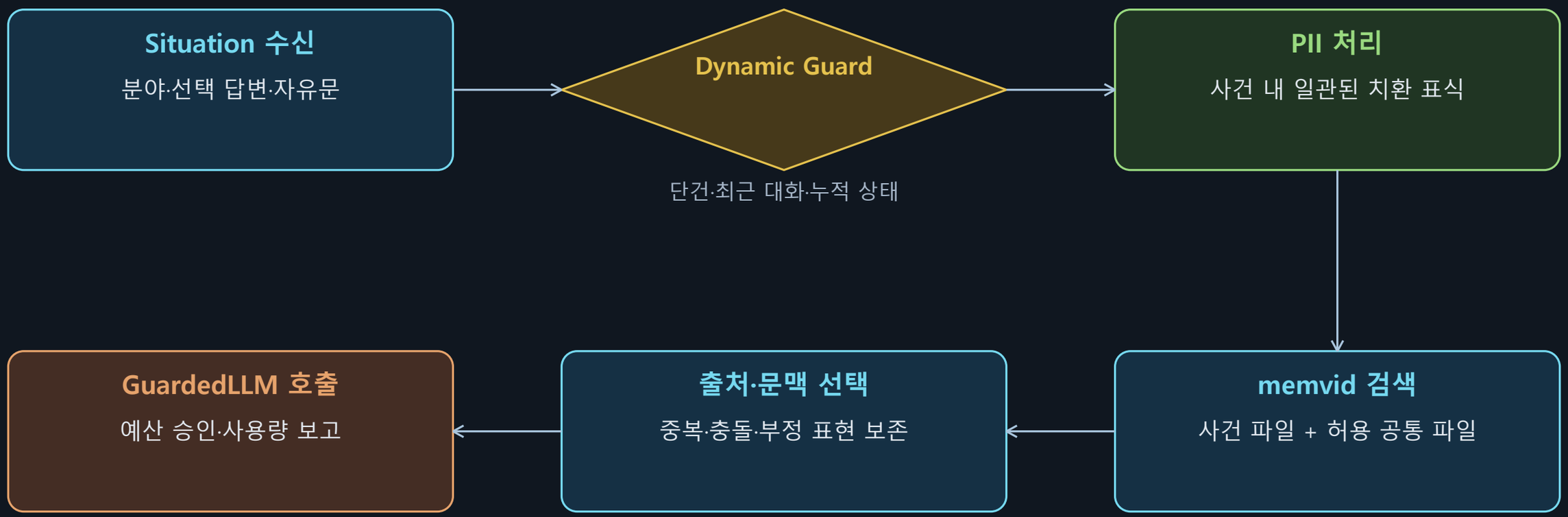
서비스를 캐시한다고 모든 사용자가 같은 파일 핸들을 공유해도 되는 것은 아닙니다. 권한 없는 사건 자료가 캐시·로그·백업에서도 반환되지 않는지 검증합니다.

MIDDLE / SCREENING

3-A · 스크리닝 전처리와 근거 검색

쉽게 말하면

개인정보를 처리한 질문으로 사건 파일에서 근거를 찾습니다. 사건 비공개 자료와 공통 공개 자료를 나누고, 출처와 함께 문맥을 선택합니다.



가드 차단이면 403으로 후속 단계를 중단합니다. k=4와 400자 분할은 원문 예시이며 memvid의 필수값이 아닙니다. 한국어 자료와 질문으로 청크·검색·모델을 평가합니다.

MIDDLE / OUTPUT POLICY

3-B · 생성 뒤에도 검사가 남는다

쉽게 말하면

검색 결과가 있다는 사실과 답변의 모든 주장이 근거로 뒷받침된다는 사실은 다릅니다. 이미 사용한 토큰은 출력 차단 여부와 별개로 정산합니다.

원문 검사·값	개정판에서의 해석
canary 유출	CANARY_LEAK 등 사유로 응답 차단·세션 정책 반영. 완전 탐지 보장은 아님
추출 예산 초과	원문 예시 +50. 누적 100 도달 시 제한하는 정책을 설정·시험
원문 복사 50% 초과	원문 예시 +35 및 요약 안내로 교체. 길이·인용 정책과 함께 검토
출처 검증	검색되지 않은 문서 ID·다른 사건 출처를 거부 또는 재검토
결과 저장·이벤트	save_report 이후 ScreeningCompleted·ConsultationSummaryDelivered 등 발행

정형 JSON 검증·근거 적합성 검증·보안 검증을 분리합니다. 의뢰인 진술을 확정 사실로 바꾸거나 존재하지 않는 판례를 생성하지 않도록 원문 연결과 변호사 검토를 제공합니다.

BOTTOM / MEMORY ADAPTER

memvid 부품: 저장·검색·임베딩 분리

쉽게 말하면

memvid는 문서와 인덱스를 파일로 관리하는 검색 부품입니다. 생성 모델과 임베딩 모델은 별도 선택이며, 파일 분리와 접근 권한도 따로 적용합니다.

역할	memvid 기준의 모듈·작업
자료 준비	문단 분할·원본 ID·페이지·버전 보존. 새 memory/corpus.py는 제안 이름
파일 어댑터	memory/memvid_store.py에서 생성·재개방·검색·종료를 모음. 제안 모듈
사건별 선택	memory/resolver.py에서 검증된 로펌·사건을 허용 파일로 매핑. 제안 모듈
임베딩	로컬·외부 제공자 중 선택. 문서와 질문에 같은 모델·차원·설정 사용
모델 실행	llm.py 등에서 근거 프롬프트·생성·usage 처리. LangChain은 선택적 연결 도구

공식 SDK의 create / use / put / find를 어댑터로 감싸면 테스트와 교체 범위를 줄일 수 있습니다. 외부 임베딩도 개인정보 전송·비용 승인 대상이며 한국어 검색 정확도를 자동 보장하지 않습니다.

BOTTOM / LEXICAL SEARCH EXAMPLE

memvid 코드: 저장 후 다시 열기

쉽게 말하면

API 키 없는 키워드 검색 학습 예제입니다. 처음 실행은 저장하고 다음 실행은 검색합니다. 운영 환경의 사건 권한 검증·암호화·동시 쓰기 통제는 별도로 필요합니다.

```
import os
from pathlib import Path
os.environ["MEMVID_OFFLINE"] = "1"
from memvid_sdk import create, use
path = Path("case-demo.mv2")
if path.exists():
    with use("basic", str(path)) as memory:
        print(memory.find("보증금", k=3, mode="lex"))
else:
    with create(str(path), enable_lex=True, enable_vec=False) as memory:
        memory.put(
            title="가상 사건 메모", label="demo", metadata={"source_id": "D001"},
            text="의뢰인은 보증금 반환을 요청했다고 진술했다.",
            enable_embedding=False,
        )
        memory.seal()
    print("저장 완료. 다시 실행하면 검색합니다.")
```

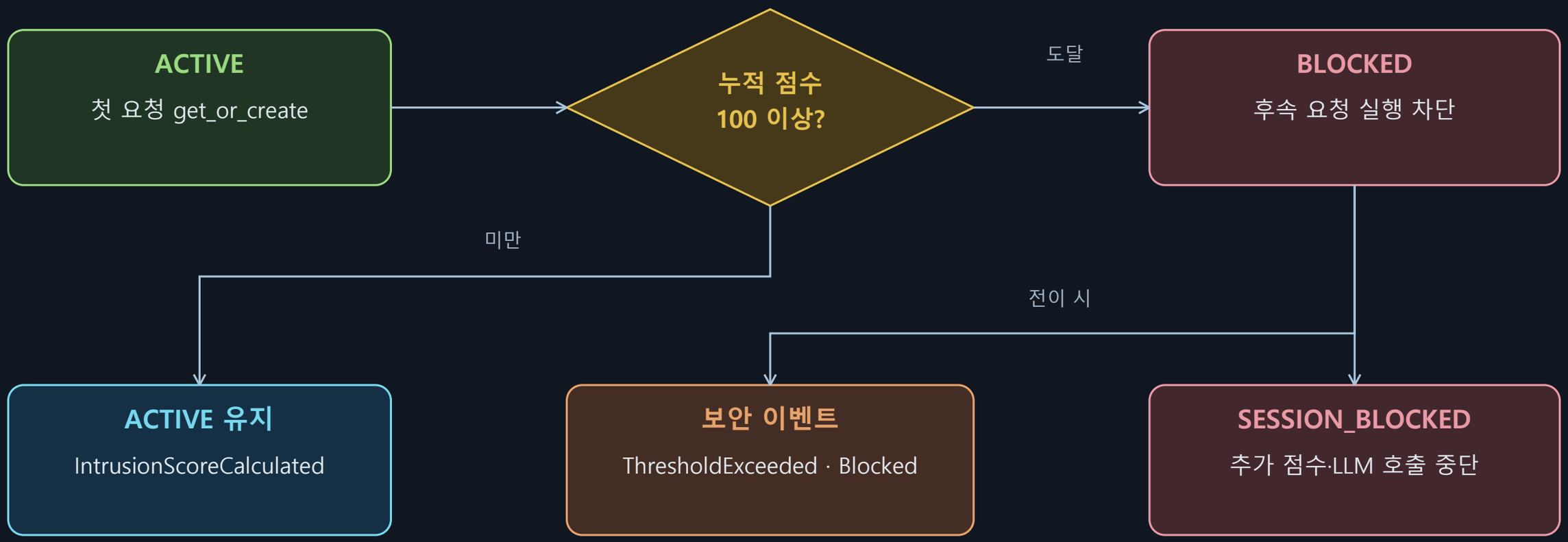
create는 기존 파일을 덮어쓸 수 있어 존재 여부를 확인합니다. 이 코드는 구문 분석만 확인했으며 SDK 실행은 미검증입니다.

MIDDLE / STATE MACHINE

4 · Dynamic Guard 상태도

쉽게 말하면

ACTIVE 상태에서 위반 점수를 누적하고 임계치에 도달하면 BLOCKED로 전이합니다. 잠긴 뒤 정상 질문이 들어와도 세션 제한 정책을 먼저 적용합니다.



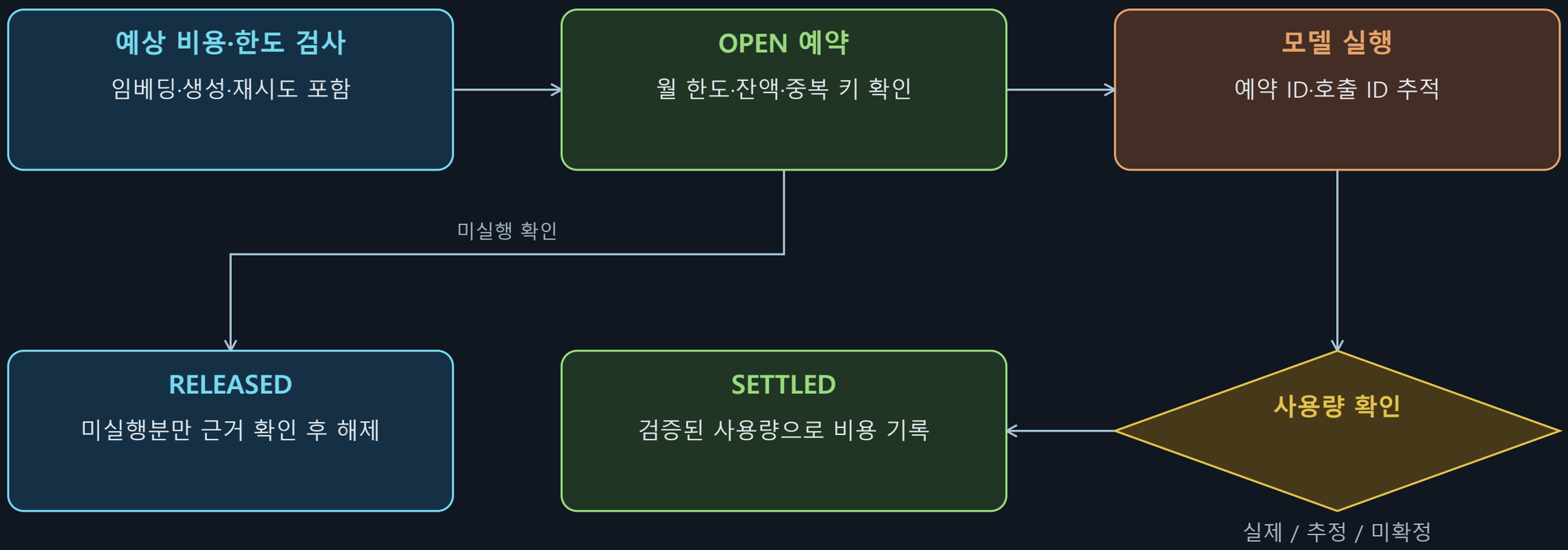
원문 가중치: 탈옥 60·주입 60·비밀 요청/대량 추출 50·길이 35. 최근 5개 메시지 결합, check·add_finding의 공통 점수 정책을 설명한 사례이며 현재 구현 결과는 재검증하지 않았습니다.

MIDDLE / COST STATE

5-A · 크레딧 예약·정산·해제

쉽게 말하면

사용 가능 잔액은 잔액에서 열린 예약을 뺀 값입니다. 예산 검사와 예약을 원자적으로 처리하고 같은 예약은 한 번만 종료합니다.



CREDIT_INSUFFICIENT(402), MONTHLY_CAP_EXCEEDED 등은 서비스 계약의 오류 코드입니다. 공급자 처리 여부가 불명확한 타임아웃을 즉시 무료로 간주하지 않습니다.

MIDDLE / COST EDGE CASES

5-B · 과금 예외를 숨기지 않기

쉽게 말하면

응답 형식 오류와 미실행은 다릅니다. 사용량을 주지 않는 공급자·가짜 모델의 결과도 0으로 숨기지 않고 추정 또는 미확정 상태로 구분합니다.

상황	기록·처리	확인할 이유
공급자 usage 제공	REPORTED · 실제 사용량 기준	제공자 내역과 대조
usage 없음	ESTIMATED 또는 미확정 추적	추정과 확정 원가 구분
모델 오류·타임아웃	미실행 확인분만 해제	이미 처리된 비용 누락 방지
JSON 파싱 실패	사용량 정산 후 LLM_INVALID_JSON	토큰은 이미 소비 가능
다른 로펌 예약 접근	RESERVATION_NOT_FOUND 등 범위 거부	다른 테넌트 원장 보호
같은 예약 중복 정산	RESERVATION_CLOSED 등 중복 거부	이중 청구·잔액 오류 방지

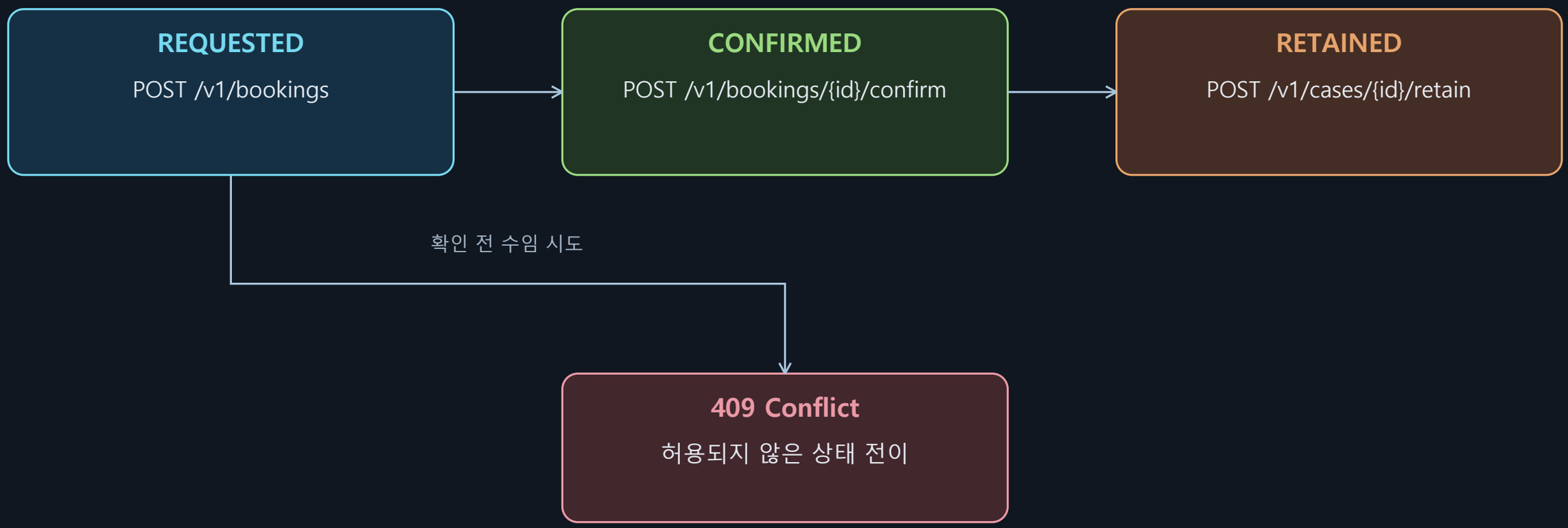
위 코드명은 원문 예시입니다. 실제 상태 코드와 영속 원장 계약을 고정하고 동시 요청·재시도·취소·부분 실행을 시험합니다.

MIDDLE / BOOKING STATE

6-A · 예약은 정해진 순서로만

쉽게 말하면

REQUESTED에서 예약을 확인하고 CONFIRMED에서 수임을 확정합니다. 순서를 건너뛴 전이는 허용하지 않으며 사건 권한과 상태를 함께 검사합니다.



이벤트: BookingRequested → BookingConfirmed → CaseRetained. 수임 요청의 fee_krw와 상태 변경은 같은 업무 계약으로 검증하며 중복 요청의 처리도 명시합니다.

MIDDLE / FUNNEL

6-B · 퍼널 지표와 null의 의미

쉽게 말하면

방문→상담→스크리닝→예약 확정→수임의 전환을 기록합니다. 데이터가 아직 없는 경우와 실제 0%를 구분하고 집계의 기간·모집단을 고정합니다.

지표	원문 식과 해석
예약 전환율	예약 확정 수 / 스크리닝 완료 수 · 집계 기간과 중복 처리 기준 명시
수임 전환율	수임 수 / 예약 확정 수 · 이벤트 누락·중복 검사
ROI	수임료 합계 / (원화로 환산한 모델 비용 + 구독료)라는 원문의 비율식
예약 1건당 비용	정산된 모델 비용 / 예약 확정 수
분모가 0	0% 대신 null. 아직 계산할 수 없음을 표현
모델 비용 확대	개정판에서는 임베딩·중간 호출 포함 여부와 비용 범위를 명시

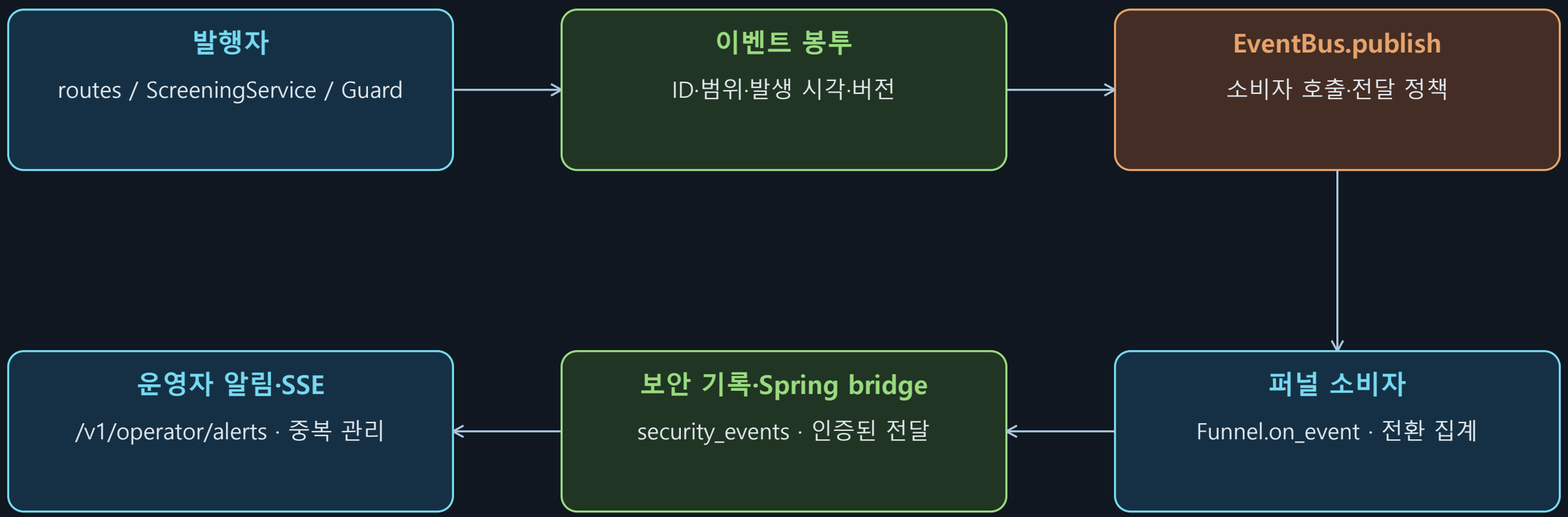
원문의 ROI 식은 매출/지정 비용 비율입니다. 일반적인 순수익 기반 투자수익률과 혼동하지 않도록 지표 정의를 화면·문서에 함께 표시합니다.

MIDDLE / DOMAIN EVENTS

7 · 도메인 이벤트: 누가 듣는가

쉽게 말하면

발행자는 구독자를 직접 알지 않도록 합니다. EventBus는 FastAPI 기본 기능이 아니라 앱의 설계이며, 영속 발행·중복 처리가 필요하면 별도 구조를 추가합니다.



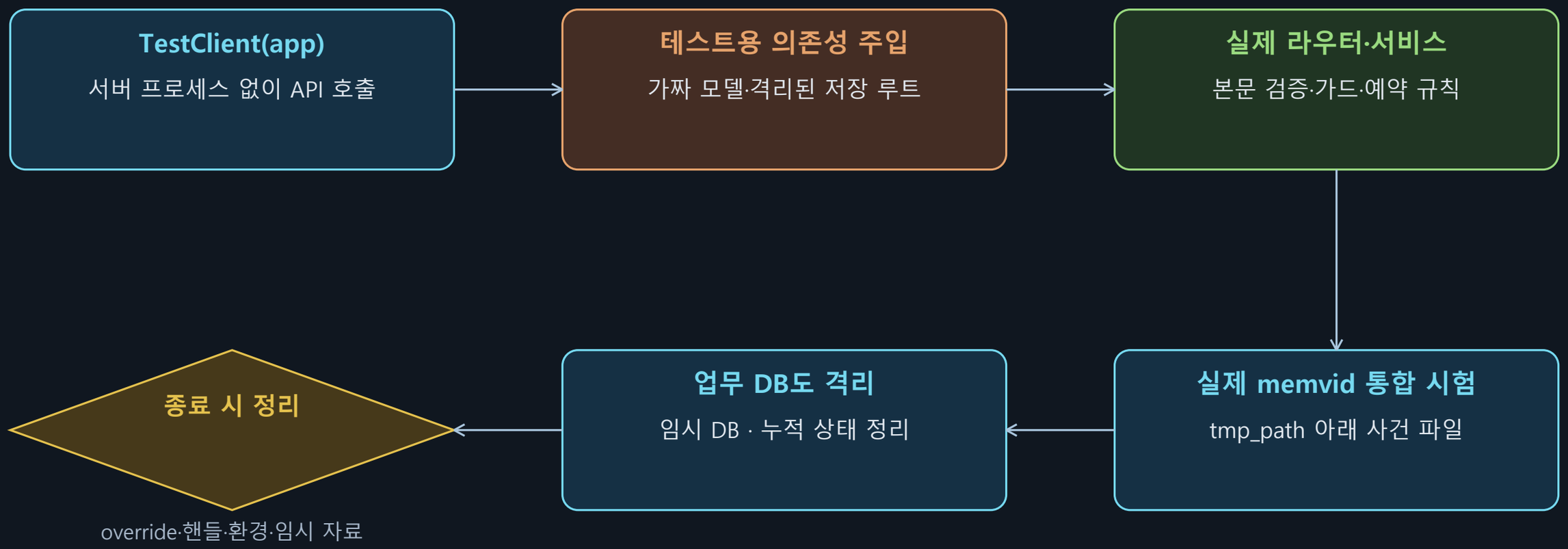
원문 이벤트 예: MarketingVisitRecorded, ConsultationRequested, ScreeningCompleted, ConsultationSummaryDelivered, IntrusionScoreCalculated, SecurityThresholdExceeded, SessionBlocked, OperatorAlertReported. 업무 DB와 발행 의도를 함께 기록하는 outbox를 검토합니다.

BOTTOM / TEST BOUNDARY

8-A · 테스트: 가장자리만 가짜로

쉽게 말하면

라우터·서비스·가드·예산 규칙은 실제 경로를 실행합니다. 유료 모델 같은 외부 경계는 대역으로 바꾸고, memvid 통합 시험에서는 임시 실제 파일로 저장·검색을 검증합니다.



FakeListChatModel·DeterministicFakeEmbedding·deps.reset은 원문의 테스트 구성 예시입니다. 개정판은 키워드 통합 시험에서 임베딩을 끄고 실제 SDK 경계를 시험할 수 있습니다. 실행했다고 주장하는 시험 결과는 아닙니다.

BOTTOM / TEST PLAN

8-B · 원문 테스트와 memvid 추가 시험

쉽게 말하면

가짜 어댑터로 서비스만 시험하는 경우와 실제 SDK 파일을 사용하는 경우를 구분합니다. 둘 중 하나만 통과했다고 운영 검색 품질까지 검증된 것은 아닙니다.

분류	시험 내용	파일·상태
함수 단위	주민번호 날짜 검증·8-gram 복사 비율	원문: test_privacy.py / test_extraction.py
보안·저장	다른 로펌 접근 거부·ID 이동 차단·SQL 주입	원문: test_storage_hardening.py / test_api.py
유스케이스	스크리닝·출처 조작·오류 전파	원문: test_screening.py
HTTP 계약	401 / 402 / 403 / 404 / 409 / 422	원문: test_api.py
memvid 통합	저장→종료→재개방→검색, 기존 파일 보존	추가 제안: test_memvid_store.py
격리·동시성	사건 A/B 누출, 핸들·잠금·캐시·삭제	추가 제안: 임시 파일 기반 통합 시험

같은 차원의 가짜 벡터를 실제 모델 파일에 섞지 않습니다. 의미 검색 품질은 실제 한국어 평가집으로 별도 측정하고 테스트 대역은 비용·분기 검증에 사용합니다.

BOTTOM / CONCEPT MAP

9-A · FastAPI 개념 ↔ 코드 읽는 자리

쉽게 말하면

책에서 개념을 읽고 해당 역할의 코드를 찾아보세요. 아래 원문 경로는 읽기 지도이며 memvid 어댑터 경로는 신규 제안으로 구분합니다.

개념	원문 예·읽는 자리	한 줄 의미
경로·메서드	routes.py · @router.post("/screenings")	URL과 함수를 연결
APIRouter·prefix	APIRouter(prefix="/v1") · include_router	주소 묶음을 앱에 등록
경로 매개변수	/screenings/{screening_id}	URL 일부를 인자로 받기
쿼리 매개변수	month: Query(pattern=...)	?month=2026-10 형식 검증
본문 모델	Situation · BookingIn · RetainIn	JSON을 Pydantic으로 검증
필드 제약	Field(ge=0) · max_length	음수·길이·범위 제한

자동 422는 본문·필드 검증 실패를 확인하는 출발점입니다. 허용하는 추가 필드와 오류 응답 모양은 명시 설정·테스트로 고정합니다.

BOTTOM / CONCEPT MAP

9-B · 응답·인증·구조를 연결하기

쉽게 말하면

프레임워크 기능과 앱의 업무 규칙을 구분하면 읽기 쉬워집니다. Depends는 연결 장치이고, 어떤 사건을 읽을 수 있는지는 우리 권한 정책입니다.

개념	원문 예·개정 연결	한 줄 의미
의존성 주입	Depends(tenant_context / get_services)	인증 문맥·실행 부품 연결
보안·인증	tenancy.py · HMAC · 운영자 역할	401 / 403 · 검증된 범위
예외·상태	HTTPException · JSONResponse	상황별 오류 코드
응답 모델	report.model_dump()	Pydantic → JSON 직렬화
스트리밍·HTML	StreamingResponse / HTMLResponse	operator.py / dashboard.py
테스트·계층	TestClient·monkeypatch·fixture / api→service→adapter	외부 경계와 업무 규칙 분리

SSE는 진행·알림을 전달하는 방식이며 작업 성공이나 비용 취소를 보장하지 않습니다. 연결 종료 후 작업 상태와 사용량을 확인하는 계약이 필요합니다.

BOTTOM / EXERCISES

연습 과제 · 직접 작게 만들어 보기

쉽게 말하면

아래는 연습 문제이며 이 문서에서 구현하지 않았습니다. 기능 하나와 그 기능을 확인할 시험을 함께 만들고, 서로 독립된 변경으로 검토합니다.

과제	확인할 결과
상태 확인	GET /v1/health/deep: DB 연결과 허용 memvid 파일의 문서/상태 확인. 파일 경로·개인정보 노출 금지
예약 필터	GET /v1/bookings?status=CONFIRMED: 자기 로펌 범위만 반환
본문 길이	RetainIn.memo에 Field(max_length=200): 201자일 때 422
상태 전이	POST /v1/bookings/{id}/cancel: REQUESTED만 허용, 나머지는 409
사건 격리	사건 A/B 파일 생성 후 권한 없는 사건의 결과가 나오지 않는지 시험
리뷰	NightCoder 등 리뷰 도구의 지적과 직접 작성한 시험 결과를 비교

개발에 독립된 작업은 분리할 수 있지만 마스킹→저장→검색→보고서처럼 선행 결과가 필요한 작업은 순서대로 진행합니다.

BOTTOM / RUN & REVIEW

실행과 검토 순서

쉽게 말하면

문서의 새 모듈 이름만으로 실행 가능한 프로젝트가 완성되는 것은 아닙니다. 실행 전 앱 엔트리포인트·의존성·설정·가상환경을 확인합니다.

순서	원문 실행법·개정 확인
1. 앱 실행	프로젝트 환경에서 uvicorn app.main:app --port 8000
2. 문서 열기	브라우저 http://127.0.0.1:8000/docs
3. 내부 인증	테스트용 자격을 준비하고 보호 API의 토큰·역할 검사 확인
4. memvid 준비	가상 자료 파일 생성, 종료 후 재개방, 키워드 검색부터 확인
5. 실패 시험	인증·입력·권한·예산·출처 오류에서 불필요한 외부 호출이 없는지 확인
6. 실제 품질	별도 한국어 평가집·사용량·격리·복구 시험 후 운영 경로 검토

실행 명령은 원문 기준입니다. 운영 키나 의뢰인 원문을 학습 예제에 넣지 않고 API 키 없는 가상 데이터 경로로 먼저 확인합니다.

APPENDIX / SOURCES

참고 근거와 변경 내역

쉽게 말하면

사용자 제공 이미지형 PDF의 0-9절과 연습 과제를 읽고 재구성했습니다. 세로 도식을 가로 벡터로 다시 그리고 저장·검색·임베딩·테스트 항목을 memvid 기준으로 개정했습니다.

자료·구분	확인할 범위
사용자 제공 원본	FastAPI 예습 노트 / 11쪽 중 1-10쪽에 내용, 마지막 1쪽은 시각적으로 빈 페이지
memvid 공식 설명	Python SDK · 임베딩 · ACL
FastAPI 공식 개념	Depends · 본문 · 테스트
이번 확인	원문 주제 대조·한글·가로 배치·흐름도·목차·코드 구문
미검증	실제 앱 수정·SDK 실행·성능·운영 보안·공급자 비용

사건별 파일은 권한 통제를 대신하지 않으며, 임베딩 모델을 바꿔도 정확도가 자동 보장되지는 않습니다. 문서에 적힌 정책·예시와 검증된 동작을 구분합니다.